Orchestration for efficient LLM Checkpointing within HPC

André Miranda
pg60238@alunos.uminho.pt
Universidade do Minho
Braga, Portugal

Gustavo Barros
pg61527@alunos.uminho.pt
Universidade do Minho
Braga, Portugal

José Soares
pg60277@alunos.uminho.pt
Universidade do Minho
Braga, Portugal

Miguel Carvalho
pg61153@alunos.uminho.pt
Universidade do Minho
Braga, Portugal

Abstract

Training Large Language Models (LLMs) in High-Performance Computing (HPC) environments is a time-consuming process that requires frequent checkpointing to ensure fault tolerance. However, concurrent writes of massive model states often saturate the shared Parallel File System (PFS), leading to significant I/O contention and training stalls. This paper proposes a policy-driven, two-tier storage orchestration system designed to mitigate the "noisy neighbor" problem in HPC clusters. By transparently redirecting checkpoints to node-local SSDs and employing a centralized orchestrator to manage asynchronous migration to the PFS via a Token Bucket algorithm, we enable granular Quality of Service (QoS) control. Our evaluation on the Deucalion supercomputer demonstrates that ... TODO

Keywords: High-Performance Computing, Deucalion, A64FX, AMD EPYC 7742, Memory Hierarchy, Large Language Models, Checkpointing, Parallel File Systems, I/O Contention, Distributed Systems, Resource Management..

1 Introduction

The rapid growth of Large Language Models (LLMs) has significantly increased the computational and storage demands of modern High-Performance Computing (HPC) systems. Training such models requires long-running, distributed jobs over clusters of GPUs or accelerators, where fault tolerance is essential to avoid losing weeks of computation due to hardware failures, network instability, or software errors. Checkpointing – the periodic saving of model states, optimizer parameters, and training metadata – has emerged as the de facto mechanism to ensure progress is preserved. However, as model sizes scale to billions or even trillions of parameters, checkpoint operations now generate massive I/O workloads, often exceeding tens to hundreds of gigabytes per save.

In shared HPC environments, these I/O-intensive workloads are typically directed to a Parallel File System (PFS), such as Lustre or GPFS, which is concurrently accessed by hundreds of users and applications. When multiple deep learning jobs attempt to write checkpoints simultaneously – as is common at the end of training epochs – the resulting synchronized I/O bursts can saturate the PFS, leading to severe contention. This phenomenon, known as the Noisy Neighbor Problem, degrades performance cluster-wide, increasing latency, reducing effective throughput, and even stalling training processes entirely. Such stalls not only waste computational resources but also disrupt the workflows of unrelated jobs sharing the same filesystem, creating a cascading effect that undermines the efficiency of the entire HPC ecosystem.

The central insight driving this work is that checkpointing is not latency-critical for persistence, but it is latency-critical for training continuation. In other words, training can resume as soon as the checkpoint is safely written to fast, temporary storage (e.g., node-local SSDs), while the durable persistence to the PFS can occur asynchronously in the background. This decoupling of checkpoint generation from long-term storage unlocks the potential to smooth I/O bursts, reduce contention, and improve overall system stability without sacrificing fault tolerance.

To address this challenge, we propose a policy-driven, two-tier storage orchestration system that combines node-local SSD staging with centralized rate control. The system is designed to transparently intercept checkpoint writes, redirect them to fast local storage, and asynchronously migrate them to the PFS under strict QoS constraints. By doing so, we minimize the stall time experienced by training jobs while regulating global I/O pressure to prevent PFS saturation.

2 Background and Motivation

2.1 Checkpointing in HPC

Checkpointing is a fault-tolerance mechanism in distributed deep learning that periodically saves the model state, including parameters, optimizer states, and gradients, to stable storage. As models scale to billions of parameters, checkpoint sizes can reach tens to hundreds of gigabytes, making I/O operations a critical bottleneck. Since large-scale training jobs may execute for hours or even weeks, failures caused by hardware faults, network instability, or software errors can lead to significant computational losses. To mitigate this problem, applications periodically save their execution state to stable storage, enabling recovery from the most recent checkpoint instead of restarting from the beginning.

2.2 Parallel File Systems and Contention

Parallel file systems are the storage backbone of HPC infrastructures, designed to provide massive data throughput

for thousands of concurrent jobs. In this work, we focus on Lustre-like file systems, which are widely deployed across TOP500 supercomputers. A typical Lustre architecture decouples metadata from file data: Metadata Servers (MDSs) manage the namespace, while Object Storage Servers (OSSs) and their underlying Object Storage Targets (OSTs) handle the heavy lifting of data read and write operations. To achieve high bandwidth, large files are typically striped across multiple OSTs, allowing compute nodes to write data in parallel via kernel-level clients using standard POSIX interfaces.

While this architecture provides massive aggregate bandwidth, it is increasingly challenged by the I/O patterns of modern Deep Learning (DL) workloads, particularly due to model checkpointing. As neural networks grow exponentially in size, saving a model's state routinely generates monolithic files or large shards ranging from tens to hundreds of gigabytes. Unlike dataset loading, which is continuous and read-intensive, checkpointing introduces periodic, extremely write-intensive I/O bursts.

When multiple large-scale distributed training jobs attempt to write these massive checkpoints to the PFS simultaneously they can saturate the OSSs and the cluster's network interconnect. This severe I/O contention not only degrades the overall storage performance for all users in the HPC cluster but also severely impacts the training efficiency itself. Because standard checkpointing mechanisms are typically synchronous, expensive GPU compute nodes are forced to stall, wasting valuable computational resources while waiting for the bottlenecked storage system to complete the massive write operations.

2.3 Quality of Service in HPC Storage

To mitigate this severe I/O contention, QoS mechanisms are increasingly necessary in HPC environments. QoS refers to the ability to control and prioritize resource allocation – such as storage bandwidth – to ensure fairness, predictability, and overall cluster stability. In the context of parallel file systems, QoS can be enforced dynamically to prevent concurrent deep learning jobs from monopolizing shared resources during their massive, synchronized checkpointing phases.

2.4 Token Bucket Algorithm

A foundational technique for enforcing bandwidth QoS is the Token Bucket algorithm. The algorithm models permission to transmit data as a bucket of tokens that is refilled over time at a configured rate. Before writing a chunk of data, the process must first accumulate enough tokens to cover the chunk size; otherwise, it waits until sufficient tokens become available. In this way, the long-term transfer rate is bounded by the token refill rate.

In the context of checkpoint migration, this mechanism is used to shape writes to the PFS and avoid sudden transfer spikes. Since unregulated checkpoint flushes can produce short, high-bandwidth bursts, the token bucket provides a simple local enforcement mechanism for rate limits assigned by the central orchestrator.

2.5 Motivation

The main motivation of this work stems from the severe I/O bottlenecks observed when training large-scale Deep Learning (DL) models on HPC clusters. As model parameters scale into the billions, their corresponding checkpoints grow to tens or hundreds of gigabytes. Currently, when distributed training jobs attempt to write these massive checkpoints to a shared PFS simultaneously.

This uncoordinated burst of I/O traffic instantly saturates the storage network, causing extreme contention. Consequently, expensive compute nodes are forced to stall while waiting for synchronous write operations to finish, wasting valuable computational time and degrading the storage performance for the entire cluster. This work is motivated by the critical need to eliminate these stalls. By developing a dynamic, QoS-aware orchestration layer, we aim to decouple checkpoint generation from PFS flushing, ensuring smooth bandwidth utilization and maximizing training efficiency.

3 System Design

As already mentioned in the motivation section, the core problem we address is the severe I/O contention caused by concurrent checkpointing in HPC clusters. To mitigate this, we propose a centralized orchestration framework equipped with dynamic QoS controls to regulate PFS access.

The main design goals of the proposed architecture are:

- **Elimination of Stalls:** Checkpoint I/O must not block the training loop. Expensive compute resources should remain fully utilized while storage operations are handled in the background.
- **Strict Traffic Shaping (Contention Avoidance):** The system must prevent "noisy neighbor" scenarios by strictly enforcing aggregate bandwidth limits, ensuring that concurrent write bursts never saturate the cluster's interconnect or the PFS Object Storage Servers.
- **Dynamic and Fair Resource Allocation:** Available storage bandwidth should be distributed intelligently among active jobs, relying on pluggable scheduling policies that consider job progress, checkpoint age, or assigned priorities.
- **Low-Latency Responsiveness:** The architecture must react instantly to cluster state transitions (e.g., a new job starting its checkpoint phase) to avoid temporary bandwidth overshoots or resource underutilization.

To achieve these goals, the system departs from traditional synchronous checkpointing. Instead of writing checkpoints directly to the PFS, the system temporarily stages

data in node-local storage (e.g., local NVMe SSDs) and asynchronously migrates it under centralized control. This decouples checkpoint generation from persistent storage operations, effectively absorbing the synchronized I/O bursts.

The architecture is built upon three primary interacting components, as illustrated in Figure 4:

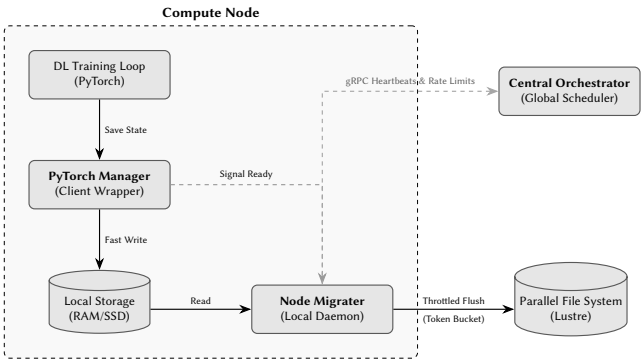

Figure 1. System architecture of the framework detailed by its components.

3.1 PyTorch Manager (Client Wrapper)

Operating directly within the deep learning application's runtime, this lightweight wrapper intercepts standard checkpointing calls (e.g., `torch.save`). Rather than blocking while data is pushed over the network to the PFS, the manager writes the checkpoint to the high-speed local storage of the compute node. Once the local write is complete, it immediately signals the node's local daemon (the Migrater) and returns control to the training loop, minimizing the computation stall time to mere seconds.

3.2 Node Migrater

The Node Migrater runs as a background process on every compute node. Upon receiving a signal from the PyTorch Manager, it takes responsibility for transferring the locally staged checkpoint to the global PFS. To reduce I/O contention on the shared PFS, the transfer speed is governed by a token bucket copy loop whose target rate is continuously updated by the Central Orchestrator.

The token bucket is initialized with an empty token balance. This avoids an initial burst at the beginning of a transfer: a Migrater cannot immediately write a large amount of data simply because a new transfer has started. Tokens are accumulated over time according to the latest rate assigned by the orchestrator, and each write chunk consumes tokens before being written to the PFS. If the assigned rate becomes zero, the Migrater pauses and keeps reporting progress, allowing the control plane to observe that the worker is stalled.

A key parameter of this mechanism is the chunk size used by the copy loop. Small chunks provide fine-grained rate enforcement and fast responsiveness to rate changes, but they increase CPU overhead and the number of write calls issued to the PFS. Large chunks reduce syscall frequency and therefore reduce write pressure, but they make the rate limiter coarser and can delay adaptation when the orchestrator changes a worker's bandwidth share. For this reason, the chunk size must balance write amplification, CPU overhead, and throughput accuracy. We evaluate this trade-off through the token bucket microbenchmark described in Section 5.4.

The Migrater also emits live transfer metrics during the copy operation. These include the number of copied bytes, remaining bytes, interval throughput, average throughput, configured rate, chunk size, and transfer timestamps. These metrics are sent through the heartbeat stream and are also used by the accumulated bandwidth plots to validate whether the aggregate transfer profile matches the orchestrator's intended bandwidth envelope.

3.3 Central Orchestrator

At the core of the system is the Central Orchestrator, which acts as the global scheduler. It maintains a thread-safe, real-time snapshot of the cluster, tracking which workers have pending checkpoints, their respective sizes, and the current training epoch. The Orchestrator applies pluggable policies (such as *Epoch-Priority* or *Age-Priority*) to calculate the optimal bandwidth share for each active node. To ensure near real-time responsiveness while minimizing network overhead, communication is handled via persistent gRPC bidirectional streams using a high-frequency heartbeat mechanism, as shown by the dashed control link in Figure 4. Rather than flooding the network with continuous push updates, the Orchestrator relies on rapid, periodic status reports from the active Migraters. Upon receiving a heartbeat containing a worker's current progress, the Orchestrator immediately recalculates the global policy and yields the updated rate limit back to the client. This efficient request-response flow over an open stream guarantees a precise enforcement of the aggregate PFS bandwidth limit, keeping latency extremely low without generating redundant control messages.

3.4 Local Storage Tier

To decouple the checkpoint generation from the global storage bottleneck, our architecture employs a two-tier storage strategy. Each compute node provides local SSD storage used as a temporary checkpoint staging area. Compared to remote PFS access, local SSDs offer dedicated, uncontended bandwidth. Initially, the core hypothesis was that staging data locally would drastically reduce the training stall time when compared to the traditional synchronous write directly to the shared PFS.

However, empirical evaluations (Figure 2) revealed a different scenario than initially anticipated. Under specific test conditions, the baseline direct-to-PFS approach actually yielded slightly lower average stall times. Yet, this baseline performance is highly sensitive to the transient background load

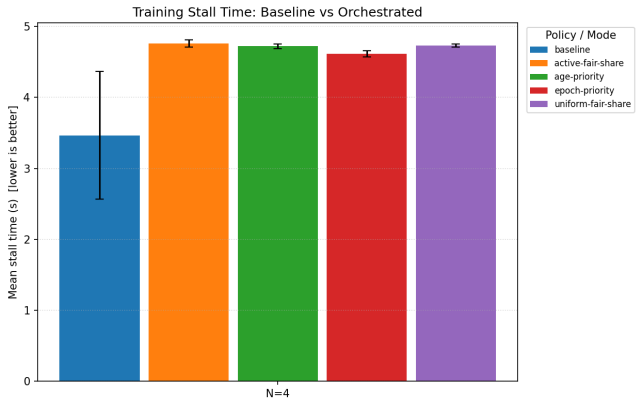

Figure 2. Comparison of stall durations during checkpointing. TODO: melhorar esta descrição Tests were conducted on Deucalion with synthetic a checkpoint workload, saving checkpoints with 5GB size. TODO: Add more details about the test conditions, such as the number of nodes, background load, partition used, ssd type and characteristics.

of the shared storage infrastructure, as evidenced by its significantly higher standard deviation.

In contrast, while the orchestrated two-tier approach (writing to local SSDs) exhibited slightly higher median stall times, it demonstrated remarkable consistency with near-zero variance. This predictability is a critical advantage for large-scale synchronous distributed training: relying on direct PFS writes exposes the job to severe latency spikes where a single delayed worker (a straggler) can force the entire cluster to stall.

Despite the lack of a drastic reduction in stall time, introducing the local storage tier remains a crucial architectural decision for two fundamental reasons:

- **Peak Control and Cluster Stability:** The local tier acts as an essential shock absorber. By capturing the instantaneous checkpoint dump locally, it empowers the Node Migrater to shape the outgoing network traffic using the Token Bucket algorithm.
- **Hardware Scalability:** While PFS bandwidth is a shared resource that inherently bottlenecks as more nodes are added to a job, local storage performance scales independently per node. As compute nodes are upgraded with faster storage technologies, the local staging time will decrease proportionally.

3.5 Control Plane

The Orchestrator is responsible for globally coordinating checkpoint migration across compute nodes. It maintains a system-wide view of:

- Active jobs
- Pending checkpoint queues
- PFS utilization
- Allocated bandwidth quotas

To manage this multi-tenant environment without introducing overhead to the compute nodes, the orchestrator acts as a centralized, stateful control plane that decouples client-side tracking from global policy execution. The framework is designed to seamlessly adapt to dynamic cluster variations – such as new job arrivals, varying checkpoint scales, and sudden job completions – by maintaining a real-time global registry of the cluster topology.

Compute nodes actively participate in this orchestration layer by emitting periodic, low-latency status updates to the control plane. Each status update encapsulates abstract operational metadata, including the worker's identity, the current size of its local staging buffer, its instant migration state, and its training progress metrics (such as the current epoch relative to the total planned epochs).

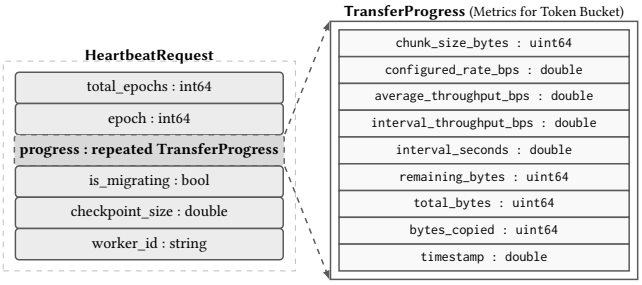

Figure 3. Representation of the structured control message sent by the Orchestrator to a compute node.

Upon receiving these updates, the control plane updates its internal representation of the cluster. dynamic job arrivals are implicitly registered on their first reported status. Conversely, job departures or unexpected worker disconnections are immediately detected, allowing the orchestrator to automatically purge the inactive resources from its scheduling pool.

To maintain structural harmony on the shared storage infrastructure and completely avoid aggregate bandwidth overshoots, the control plane employs a reactive, push-on-event model. Any state change that alters the cluster configuration or changes a worker's buffer depth triggers an immediate re-evaluation of the global storage bandwidth. The orchestrator examines the entire cluster state as a unified topology, executes a pluggable scheduling policy, and calculates revised bandwidth quotas for all active nodes simultaneously. New explicit actions (such as initiating a flush, dynamically modifying the allowed transfer rate, or placing a migration on hold) are immediately pushed back to the compute nodes. This global coordination loop ensures that node-local data transfers adapt instantly to multi-tenant cluster shifts.

Furthermore, the orchestrator ensures strict resource isolation and multi-tenant fairness across parallel jobs by enforcing an aggregate storage bandwidth ceiling.

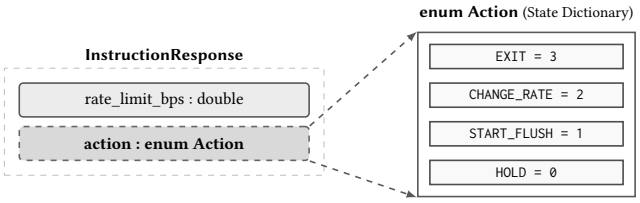

Figure 4. Example of a control message sent by the Orchestrator to a compute node. This structured message allows the Orchestrator to communicate the control commands to the Node Migraters, enabling dynamic adjustments to their behavior based on the current cluster state.

4 Implementation

4.1 Token Bucket Model

The token bucket is implemented as a local copy primitive used by the Migrater. The function receives a source path, a destination path, a throughput limit, and a chunk size. The throughput limit can be either a fixed value or a dynamic callback, allowing the orchestrator to change the assigned bandwidth while a checkpoint is already being copied.

At runtime, the copy loop repeatedly samples the current rate, refills the token balance according to elapsed time, and only performs a read/write operation when enough tokens are available for the next chunk. The bucket starts empty to avoid startup bursts. When there are not enough tokens for the next chunk, the Migrater computes the missing token deficit and derives the required waiting time from the configured rate. This sleep is capped to keep the loop responsive to rate changes issued by the orchestrator. If the configured rate becomes zero, the token balance is reset, the loop pauses in short intervals, and progress is still reported so that the orchestrator can observe stalled transfers.

The implementation also records per-transfer metrics. For each reporting interval, it computes interval throughput, average throughput, bytes copied, bytes remaining, the configured rate at report time, and the active chunk size. These metrics are used both for online monitoring and for offline evaluation of bandwidth enforcement.

4.2 Scheduling Policy

The scheduling policies are implemented following a standard interface; these are responsible for guiding the "orchestrator" to transmit the responses of 4 possible actions (`START_FLUSH`, `CHANGE_RATE`, `HOLD`, and `EXIT`). A `CHANGE_RATE` is sent whenever a policy assigns a `rate_limit_bps` to a worker that is already flushing; the token bucket in the migrator rereads the rate at runtime and adjusts it without needing a restart. This allows the policies to be chosen via a command-line flag, providing greater versatility during testing. A worker is considered active whenever the `checkpoint_size` in the last heartbeat is greater than 0. Policies that take activity into account send a `HOLD` message to currently inactive workers, and the bandwidth is only distributed among active workers.

The state of each worker, maintained in the orchestrator's `ClusterState` and updated at every heartbeat, is represented by the `WorkerState` structure. Table 1 summarizes its fields, indicating the origin of each one and the policies that consume them.

Field	Source	Used by
`worker_id`	`socket.gethostname()` (migrater)	static-priority
`checkpoint_size`	bytes pending flush	active, static, age
`is_migrating`	migrater flag	(reserved)
`last_seen`	`time.time()` on update	—
`pending_since`	`time.time()` on transition $0 \rightarrow >0$	age-priority
`epoch`	progress reported by the migrater	epoch-priority
`total_epochs`	total epochs reported by the migrater	epoch-priority

Table 1. Fields of the `WorkerState` maintained by the orchestrator.

4.2.1 NoLimit.

The baseline policy always returns `START_FLUSH` with the `rate_limit_bps` fixed at 1GB/s, regardless of the number of workers, the cluster state, or the provided PFS bandwidth. Since it has no notion of activity, all existing workers receive bandwidth even if they are not currently active.

4.2.2 FixedRate.

The policy gives each worker a fixed rate of r, regardless of how many workers exist and how many are active:

$$r_i = r = \text{const}, \quad \forall i \in W, \tag{1}$$

where r_i is the rate assigned to worker i, r is the configured constant, and W is the set of workers registered in the orchestrator.

4.2.3 UniformFairShare.

The policy divides the defined bandwidth equally among all registered workers, whether they are active or not:

$$r_i = \frac{B_{\text{PFS}}}{|W|}, \quad \forall i \in W, \tag{2}$$

where r_i is the rate assigned to worker i, B_{PFS} is the aggregate PFS bandwidth ceiling (`-pfs-bw`), and $|W|$ is the total number of registered workers. The policy ensures that the sum of all r_i is equal to the PFS bandwidth value, eliminating the violation of this defined ceiling. The problem with this policy is that it wastes bandwidth on workers that are not using it because they are inactive.

4.2.4 ActiveFairShare.

The ActiveFairShare policy is an upgrade of UniformFairShare, it has the same foundation, but divides the bandwidth only among active workers:

$$r_i = \begin{cases} \dfrac{B_{\text{PFS}}}{|A|}, & \text{if } i \in A, \\ 0 \text{ (HOLD)}, & \text{if } i \notin A, \end{cases} \tag{3}$$

where r_i is the rate assigned to worker i, B_{PFS} is the aggregate PFS bandwidth ceiling, and $A \subseteq W$ is the subset of active workers (with `checkpoint_size` > 0). In other words, bandwidth is not wasted on workers that are currently inactive. Since workers become active and inactive, whenever a change of this kind occurs, the bandwidths must be adjusted via the `CHANGE_RATE` instruction.

4.2.5 StaticPriority. This policy allocates bandwidth following weights that give workers certain degrees of priority. These weights are defined in a JSON file, and then the bandwidth is calculated by the following formula:

$$r_i = B_{\text{PFS}} \cdot \frac{w_i}{\sum_{j \in A} w_j}, \quad i \in A, \tag{4}$$

where r_i is the rate assigned to worker i, B_{PFS} is the aggregate bandwidth ceiling, A is the subset of active workers, and $w_i > 0$ is the weight assigned to worker i in the priority file (workers missing from the map fall back to a configurable `default_priority`). In this policy, unlike the others, the orchestrator does not decide which worker deserves more bandwidth; this is decided by a file created by someone.

4.2.6 AgePriority. This dynamic policy combines two signals: the pending checkpoint size (those with more pending bytes finish faster if given more bandwidth, thus freeing up the system) and the time a worker has been waiting to perform the checkpoint (anti-starvation). Since the signals are of different magnitudes (bytes and seconds), normalization was used to relate them and assign a priority value.

Each signal is normalized to $[0, 1]$ by the maximum observed in the active set:

$$\tilde{s}_i = \frac{s_i}{\max_{j \in A} s_j}, \qquad \tilde{a}_i = \frac{a_i}{\max_{j \in A} a_j}, \tag{5}$$

where $s_i = \texttt{checkpoint\_size}_i$ is the number of pending bytes and $a_i = t_{\text{now}} - t_i^{\text{pending}}$ is the age of the pending state (in seconds), with t_i^{pending} being the instant worker i transitioned from inactive to active.

The priority results from a convex combination of the two normalized signals, lower-bounded by a numerical floor to prevent priorities from degenerating to zero:

$$p_i = \max\big(\alpha \cdot \tilde{s}_i + \beta \cdot \tilde{a}_i,\ \varepsilon\big), \tag{6}$$

with $\alpha, \beta \geq 0$, $\alpha + \beta > 0$, and $\varepsilon = 10^{-6}$. The final rate is assigned proportionally:

$$r_i = B_{\text{PFS}} \cdot \frac{p_i}{\sum_{j \in A} p_j}, \quad i \in A, \tag{7}$$

where r_i is the rate assigned to worker i and B_{PFS} is the aggregate bandwidth ceiling.

The `ClusterState` is responsible for updating the wait time. It is possible to vary the importance of each metric by varying the pair (α, β): if $\alpha = 1$ and $\beta = 0$, only the `checkpoint_size` value will be considered; if $\alpha = 0$ and $\beta = 1$, only a worker's wait time value will be taken into account, that is, a purely anti-starvation policy. The balanced value, giving equal attention to both signals, is $\alpha = 0.5$ and $\beta = 0.5$.

4.2.7 EpochPriority. This dynamic policy uses a signal originating from the training side of the model. The migrator sends `epoch` and `total_epochs` at every heartbeat, which are collected by the orchestrator.

The priority is calculated by a linear-with-floor function of the training progress:

$$\rho_i = \text{clip}\left(\frac{e_i}{E_i},\ 0,\ 1\right), \tag{8}$$

$$p_i = \phi + (1 - \phi) \cdot \rho_i, \tag{9}$$

$$r_i = B_{\text{PFS}} \cdot \frac{p_i}{\sum_{j \in A} p_j}, \quad i \in A, \tag{10}$$

where e_i is the current epoch reported by worker i, E_i is its total number of epochs, $\rho_i \in [0, 1]$ is the normalized progress, $\phi \in [0, 1]$ is the configurable priority floor (`-epoch-floor`, with a default value of $\phi = 0.2$), p_i is the worker's final weight, r_i is the assigned rate, and B_{PFS} is the aggregate bandwidth ceiling.

The goal with this policy is that jobs closer to completing their training have already used more computation and have fewer future opportunities to protect themselves through checkpoints. In this way, they are better protected from contention than jobs that have just started. To control the weights of recently started jobs, the configurable parameter ϕ is also passed, with a default value of 0.2, to ensure that jobs at the beginning of their progress (i.e., $\rho = 0$) receive a fraction of the bandwidth and do not experience total starvation.

Table 2 summarizes all implemented policies, indicating their type, the signals they consume, and the specific *flags* for each one. All of them additionally accept `-pfs-bw` to define the aggregate PFS bandwidth ceiling.

Policy	Type	Signals used	Own flags
`no-limit`	baseline	—	—
`fixed-rate`	static	—	(`-pfs-bw`)
`uniform-fair-share`	static	no. of workers	`-pfs-bw`
`active-fair-share`	static	no. of active workers	`-pfs-bw`
`static-priority`	static	weights from JSON	`-priority-map`
`age-priority`	dynamic	`checkpoint_size`, `pending_since`	`-alpha`, `-beta`
`epoch-priority`	dynamic	`epoch` / `total_epochs`	`-epoch-floor`

Table 2. Summary of the implemented scheduling policies.

5 Evaluation

Our evaluation seeks to answer the following architectural and behavioral questions:

- **Stall Elimination:** Can the two-tier staging strategy decouple the training loop from persistent storage, reducing or eliminating synchronous application stalls?

- **Global QoS Enforcement:** Can the central orchestrator accurately enforce collective bandwidth constraints across concurrent deep learning jobs without causing PFS saturation?
- **Token Bucket Parameterization:** Which chunk size provides a suitable trade-off between throughput accuracy, responsiveness to rate changes, and reduced write pressure on the PFS?
- **Policy Effectiveness:** How do different pluggable algorithms (*Active Fair Share*, *Age-Priority*, and *Epoch-Priority*) dynamically distribute storage throughput under resource contention?

5.1 Experimental Testbed

All experiments were conducted on the **Deucalion** supercomputer, specifically leveraging the homogeneous **ARM compute partition**. Each compute node within this partition is equipped with a Fujitsu A64FX processor featuring 48 cores (plus 4 assistant cores) based on the ARMv8.2-A SVE architecture, alongside 32 GiB of High Bandwidth Memory (HBM2) providing an internal bandwidth of up to 1024 GB/s. The nodes run Rocky Linux 8.5 and are interconnected via a high-speed **NVIDIA Mellanox InfiniBand HDR100** network fabric.

For the intermediate storage tier, each compute node utilizes a portion of its local high-speed **NVMe SSD** storage layer, which acts as the localized staging target to absorb initial write spikes. The centralized, production persistent layer is a shared **Lustre Parallel File System** managed by an infrastructure consisting of 8 Metadata Server (MDS) nodes backing 430 TB of hot pools, and 32 Object Storage Server (OSS) nodes backing 10 PB of aggregate capacity, yielding a collective storage throughput ceiling of up to 340 GB/s for read operations and 260 GB/s for write operations.

5.2 Methodology and Execution Setups

The control plane is deployed with the central orchestrator executing on a dedicated, reachable compute node within the cluster, while an independent instance of the node-local (*Node Migrater*) is co-located on each active worker node. Communication between components is sustained via high-frequency bidirectional gRPC streams, exchanging telemetria packets at a static sampling rate of 0.1 seconds.

Across all experimental scenarios, workloads are evaluated under two structural environments:

- **Baseline:** Represents the traditional, uncoordinated checkpointing workflow. Training processes bypass any intermediate layers and write model state archives directly to the shared Lustre PFS concurrently using synchronous `torch.save` primitives.
- **Orchestrated:** Represents the proposed framework where storage interactions are transparently redirected to the high-speed local staging tier. Upon local write completion, the training process immediately returns to execution, while background data daemons migrate the staged checkpoints to the PFS, regulated in real-time by dynamic token-bucket rate limits dictated by the central orchestrator.

5.3 Benchmarks and Workloads

To evaluate the system under realistic deep learning workloads, we developed an automated end-to-end benchmarking framework located under `benchmarks/e2e/`, consisting of simulated workload orchestrators, baseline drivers, and stateful metrics collectors. The execution states simulate parallel jobs competing for a predefined collective storage bandwidth bottleneck. We evaluate system performance under three programmatic traffic paradigms:

- **Burst Traffic Workload:** Evaluates peak absorption and traffic-shaping agility. All registered worker nodes are forced to trigger massive checkpoint save operations at the exact same logical timestamp (simulating a synchronized training epoch boundary), stressing the local staging tier's shock-absorbing capabilities and the token bucket's throttling precision.
- **Steady Traffic Workload:** Evaluates long-term stability and resource isolation. Multiple job instances run in parallel, triggering checkpoint operations at staggered, regular intervals to test how the orchestrator maintains smooth global bandwidth profiles over time.
- **Dynamic Churn Workload:** Evaluates control plane responsiveness and convergence speed. Job instances are initiated with dynamic, time-staggered arrival and departure vectors. This churn triggers continuous re-evaluations within the orchestrator, forcing the scheduling policies to dynamically compress or expand the active bandwidth shares of running tasks on-the-fly.

The training loop metadata replicates the lifecycle of LLM distributed checkpointing. Each individual checkpoint save operation dumps a payload size of 5 GB representing model weights, gradients, and optimizer states. The global persistent bandwidth allocated to the orchestrator ceiling is set to 2 GB/s to simulate high-contention saturation boundaries. Telemetry and effective aggregate network throughput curves are continuously written to structured metric logs (`.jsonl` format) across all evaluating configurations.

5.4 Token Bucket Microbenchmark

Before evaluating the full orchestration system, we benchmarked the token bucket copy primitive in isolation. The goal was to identify a chunk size that keeps the effective throughput close to the configured rate while reducing the number of write operations issued to the PFS. This is important because the token bucket is not only a rate limiter: it

also determines the granularity at which data is written to the shared storage system.

The benchmark sweeps multiple configured throughput targets and chunk sizes. For each configuration, it runs the token bucket copy both against node-local storage and against the PFS. The benchmark records the expected transfer time, actual transfer time, expected throughput, actual throughput, the actual/expected throughput ratio, and PFS/local time and throughput ratios. It also records the number of concurrent SLURM jobs visible during the experiment, so that results can be interpreted under the observed cluster load.

The plotting pipeline consumes the JSONL benchmark output and generates heatmaps for actual throughput, actual/expected throughput ratios, and PFS/local ratios, as well as line plots showing throughput ratio by chunk size. These plots are used to compare local and PFS targets and to identify chunk sizes that preserve throughput while reducing write frequency.

Figure 5 shows the PFS actual/expected throughput ratio as a function of configured throughput for different chunk sizes. Very small chunks provide fine-grained control, but they require many write operations and show lower throughput at high configured rates. Larger chunks reduce the number of writes, but overly large chunks can make the limiter less responsive. In our ARM/PFS experiments, the 4 MiB chunk size provided the best balance: it maintained a high actual/expected throughput ratio over a broad range of configured rates while substantially reducing the write count compared with smaller chunks.

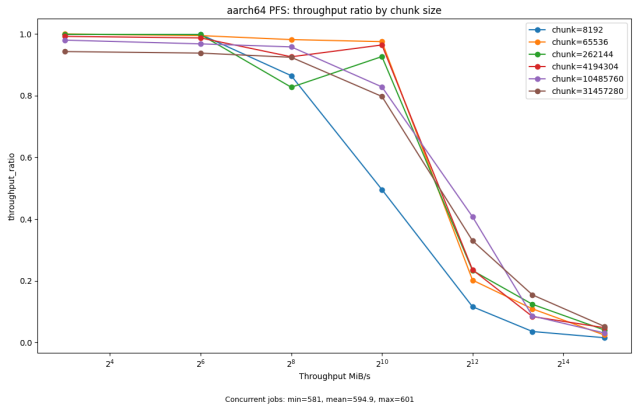

Figure 5. Token bucket actual/expected throughput ratio on the PFS for different chunk sizes and configured throughput targets. The 4 MiB chunk size provides a practical balance between throughput accuracy and reduced write frequency.

Figure 6 compares the PFS throughput against the local-storage baseline for the same configurations. The results confirm that the PFS can behave differently from local storage, especially at higher configured rates and larger chunk sizes, which motivates tuning the token bucket parameters on the actual target storage system rather than relying only on local-node behavior.

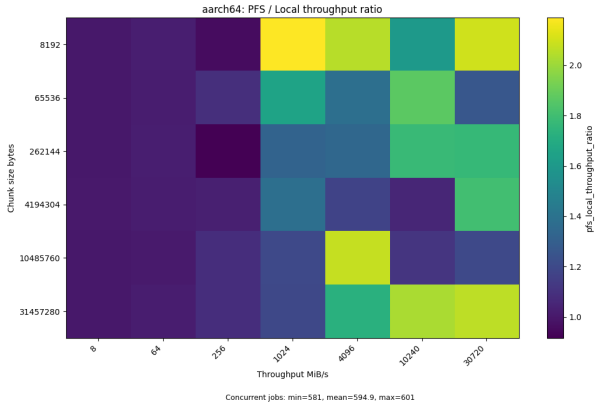

Figure 6. PFS/local throughput ratio for token bucket transfers across chunk sizes and configured throughput targets. Values above 1 indicate configurations where the PFS achieved higher throughput than the local baseline.

5.5 Metrics

- Checkpoint latency
- Training stall duration
- PFS throughput and variability
- Fairness index across jobs
- I/O congestion
- CPU/GPU and memory usage
- Token bucket actual/expected throughput ratio
- PFS-over-local throughput and time ratios

6 Related Work

The optimization of checkpointing in HPC has been extensively studied, primarily focusing on reducing write latency and mitigating PFS congestion.

6.1 Architectural and Tiering Approaches

Several systems utilize hierarchical storage to hide I/O latency. **Check-N-Run** [1] focuses on staging checkpoints in local node resources (DRAM or SSDs), allowing training to resume immediately while data is moved to the PFS in the background. However, it lacks a global coordination mechanism to prevent PFS saturation during the background flush. **DeepFreeze** [4] (similar to Monarch) employs transparent storage tiering to accelerate the write path by abstracting complex local tiers like NVMe or PMEM from the user. While effective at the node level, these systems often ignore the "noisy neighbor" effect in shared environments.

6.2 Adaptive and Optimization Techniques

Other research focuses on the frequency and size of checkpoints. **CheckFreq** [3] introduces an algorithmic approach

to identify optimal checkpoint boundaries and pipelines the process with training iterations, dynamically adjusting frequency to maintain overhead below a threshold. **BitSnap** [5] addresses the data volume by employing sparsification and quantization, reducing checkpoint sizes from 16-bit to 4-bit weights before they reach the I/O layer. While these reduce total I/O pressure, they do not manage the storage hierarchy or provide holistic QoS guarantees.

6.3 Software-Defined Storage and QoS Control

Our work is heavily informed by **PADLL** [2], a storage middleware that enables application-agnostic QoS control for both data and metadata workflows. PADLL utilizes a decoupled Software-Defined Storage (SDS) architecture, where data plane stages intercept POSIX calls and a centralized control plane coordinates I/O rates across all jobs. While PADLL is a general-purpose QoS framework, our system specializes this paradigm for the specific lifecycle of LLM checkpoints – integrating node-local SSD staging with PADLL-inspired Token Bucket rate limiting to ensure cluster harmony during massive model state migrations.

7 Conclusions

Our project investigated the impact of checkpoint-induced I/O contention in shared HPC environments and proposed a policy-driven storage orchestration framework to improve the stability and scalability of distributed Deep Learning training workloads. The presented architecture combines node-local SSD staging with centralized QoS-aware bandwidth control, enabling checkpoints to be persisted asynchronously while limiting pressure on the PFS.

The evaluation conducted on the Deucalion supercomputer demonstrates that the proposed system effectively smooths checkpoint traffic and prevents aggregate PFS saturation under synchronized burst scenarios. Although the local staging layer did not always reduce the absolute checkpoint stall time compared to the baseline direct-to-PFS approach, it provided significantly more predictable and stable behavior, eliminating the large latency variance caused by transient storage contention. This predictability is especially important for distributed training environments, where a single delayed worker can stall the entire job.

The experiments further showed that the centralized orchestration model can accurately enforce global bandwidth limits while dynamically adapting to job arrivals, departures, and checkpoint queue variations. The token bucket mechanism proved effective at shaping storage traffic, and the microbenchmark analysis identified a 4 MiB chunk size as a practical balance between throughput accuracy, responsiveness, and reduced write pressure on the PFS.

Overall, the proposed architecture demonstrates that combining node-local storage staging with centralized, policy-driven QoS enforcement is a viable and effective strategy for improving the robustness and scalability of checkpointing in modern HPC systems. Beyond reducing the impact of the noisy neighbor problem, the framework establishes a foundation for storage-aware scheduling and fine-grained resource orchestration tailored to the unique I/O behavior of Large Language Model training workloads.

8 Future Work

While the proposed policy-driven, two-tier storage orchestration system successfully mitigates parallel file system congestion and eliminates training stalls, our prototype evaluation highlights several open paths for future research and infrastructural optimization.

- **Distributed and Fault-Tolerant Orchestrator:** The current centralized control plane introduces a potential scalability bottleneck and a single point of failure. Future work should explore distributed or hierarchical orchestration designs to improve resilience and enable operation at larger cluster scales.
- **Advanced Scheduling Policies:** The current framework supports pluggable QoS policies, but more expressive scheduling strategies could be incorporated, such as deadline-aware, congestion-aware, or learning-based policies that dynamically adapt to workload behavior and cluster state.
- **Integration with Native PyTorch Asynchronous Checkpointing:** Tighter integration with PyTorch's native asynchronous checkpointing APIs and distributed training frameworks (e.g., FSDP or DeepSpeed) would improve usability, reduce overhead, and enable broader adoption in existing LLM training pipelines.
- **Evaluation with Real LLM Training Workloads:** Future evaluations should extend beyond synthetic checkpoint workloads to full end-to-end LLM training runs in order to better quantify impacts on training throughput, convergence behavior, and system-level efficiency.
- **Exascale and Multi-Tenant Stress Testing:** Additional experiments at larger scale are required to assess the system's robustness under extreme contention scenarios involving hundreds or thousands of concurrent checkpointing jobs.
- **Adaptive Storage-Aware Control Loops:** Incorporating real-time feedback from the storage backend (e.g., PFS queue depth or OSS utilization) could enable more adaptive control policies that react directly to system-level congestion signals.
- **Support for Multi-Tier and Burst-Buffer Architectures:** Extending the current two-tier design to support additional storage layers such as burst buffers or persistent memory would allow more flexible data placement strategies in heterogeneous HPC systems.

- **Checkpoint Compression and Deduplication:** Reducing checkpoint size through lightweight compression or deduplication techniques could further alleviate pressure on both local staging storage and the shared PFS, complementing the proposed QoS mechanisms.

References

[1] Assaf Eisenman et al. 2022. Check-N-Run: A Checkpointing System for Training Deep Learning Recommendation Models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 929–943. https://www.usenix.org/conference/nsdi22/presentation/eisenman

[2] R. Macedo et al. 2023. PADLL: Taming Metadata-intensive HPC Jobs Through Dynamic, Application-agnostic QoS Control. *arXiv preprint arXiv:2302.06418* (2023). https://arxiv.org/abs/2302.06418

[3] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 203–216. https://www.usenix.org/conference/fast21/presentation/mohan

[4] Bogdan Nicolae, Jian Li, Justin Wozniak, George Bosilca, Matthieu Dorier, and Franck Cappello. 2020. DeepFreeze: Towards Scalable Asynchronous Checkpointing of Deep Learning Models. In *CCGrid 2020*. 172–181. doi:10.1109/CCGrid49817.2020.00076

[5] Y. Peng et al. 2025. BitSnap: Checkpoint Sparsification and Quantization in LLM Training. *arXiv preprint arXiv:2511.12376* (2025). https://arxiv.org/abs/2511.12376

Received 25 May 2026